

《计算机系统基础》重点知识及方法总结

刘鼎琨 于珞珈山

目录

- 第二章：信息的表示和处理 2
 - 一、 信息存储概览 2
 - 二、 布尔代数 3
 - 三、 整数 3
 - 四、 浮点数 6
- 本章总结 8

第二章：信息的表示和处理

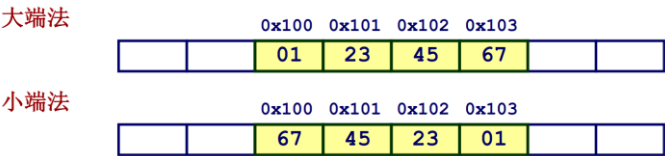
一、信息存储概览

- (1) 进制相关：
 - 计算机采用二进制，主要由其电气特性决定。
注：电学上允许计算机采用多进制，但在某些方面反而不如二进制出色。
 - 需要熟悉进制转化。
- (2) 字数据大小：

C Data Type	Typical 32-bit	Typical 64-bit	x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	8	8
float	4	4	4
double	8	8	8
pointer	4	8	8

注：i. 64 位机中正常情况下 long 应该为 8 位，不是 4 位。
ii. 除此之外，还有 int32_t (uint32_t)和 int64_t(uint64_t)等类型。
iii. 如 short 类型，包含了两个字节，等同于 4 位的十六进制码。
iv. 基本换算：1byte = 8bits 1KB = 1024byte 1MB = 1024KB
1G = 1024MB 1TB = 1024G 1PB = 1024TB 1EB = 1024PB = 2⁶⁰ byte

- (3) 字节排序：
 - 大端法: Sun, PPC Mac, Internet
最低有效字节具有最高地址。
 - 小端法: x86, ARM processors running Android, iOS, and Windows
最低有效字节具有最低地址。



- (4) 表示字符串： 每个字符以 ASCII 格式编码
 - 标准的 7 位字符集编码

字符	十进制 ASCII 码	十六进制 ASCII 码
“\0”	000	0x00
“ ”	032	0x20
“0”	048	0x30
“A”	065	0x41
“Z”	090	0x5A
“a”	097	0x61
“z”	122	0x71

二、 布尔代数

(1) C 语言中的位级运算：

&交 |并 ^异或 ~补

注：区别于逻辑运算（&&与 ||或 !非），涉及提前终止。

(2) C 语言中的移位运算：

➤ 左移运算: $x \ll y$

将位向量 x 左移 y 位，在左边把多余的位丢掉，右边补上 0。

➤ 右移运算: $x \gg y$

将位向量 x 右移 y 位，在右边把多余的位丢掉，

✧ 若为**逻辑右移**，在左边补上 0

✧ 若为**算数右移**，在左边补最高有效位的值

➤ 未定义行为

移位数量 < 0 或者 $\geq$ 字长

注： $2^8=256$; $2^{16}=65,536$; $2^{31}=2,147,483,648$

三、 整数

(1) 无符号数和有符号数的编码：

$$B2U(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i \quad B2T(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

注：对于补码的计算，除却上述高位变负的定义外，还可以通过取反加 1 的方式，不过这一情况不永远成立。 $x = TMin$ 属于标准反例。

C数据类型	最小值	最大值
[signed]char	-128	127
unsigned char	0	255
short	-32 768	32 767
unsigned short	0	65 535
int	-2 147 483 648	2 147 483 647
unsigned	0	4 294 967 295
long	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
unsigned long	0	18 446 744 073 709 551 615
int32_t	-2 147 483 648	2 147 483 647
uint32_t	0	4 294 967 295
int64_t	-9 223 372 036 854 775 808	9 223 372 036 854 775 807
uint64_t	0	18 446 744 073 709 551 615

➤ 相应的关系：

✧ 无符号整数： $Umin = 0$; $Umax = 2^w - 1$;

✧ 补码： $Tmin = -2^{w-1}$; $Tmax = 2^{w-1} - 1$

✧ $|Tmin| = Tmax + 1$;

✧ $Umax = 2 * Tmax + 1$;

➤ 相关调用：

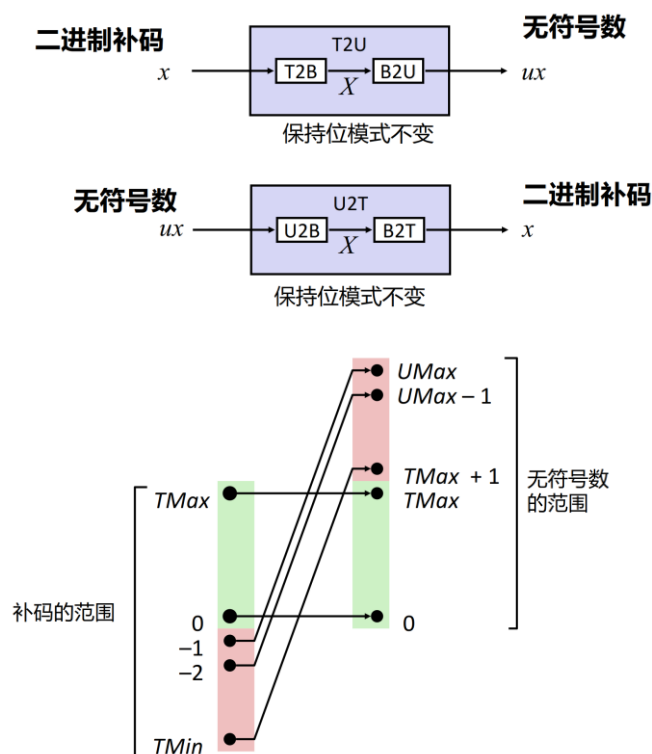
✧ 利用 `#include <limits.h>` 声明常量，例如 `ULONG_MAX`, `LONG_MAX`, `LONG_MIN` 等。

(2) 类型转化：

① 分类：强制类型转换、赋值和过程调用中的隐式的强制类型转换。在表达式中，有符号数被隐式的强制类型转换为无符号数。

注：常量默认情况下被认为是符号数，如果后缀为“U”，则为无符号数。

② 特点：位级模式不变，解读方式产生差异。



注：i. 此处可以通过补码的定义进行理解。

ii. 易错的系统 Bug: `sizeof(int)` `size_t` `unsigned i;`
`void *memcpy(void *dest, void *src, size_t n);`
`void *malloc(size_t size);`

(3) 拓展：

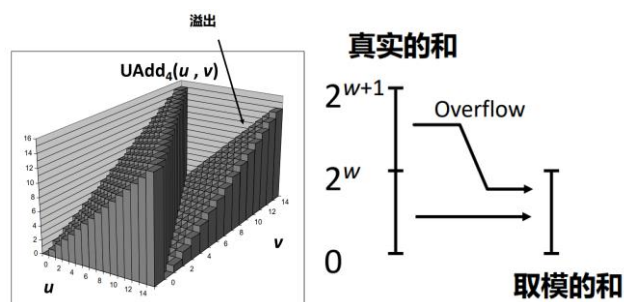
- 无符号数：0 拓展，在最前面加 0.
- 有符号数：符号位拓展.
- C 语言默认符号拓展，这样具体数值可以保持不变.

(4) 截断：

- ① 无符号数：等价于模(mod)运算.
- ② 有符号数：类似模(mod)运算.
 - ✧ 若符号位不改变，值不变.
 - ✧ 若符号位变化，相当于取模.

(5) 加法：

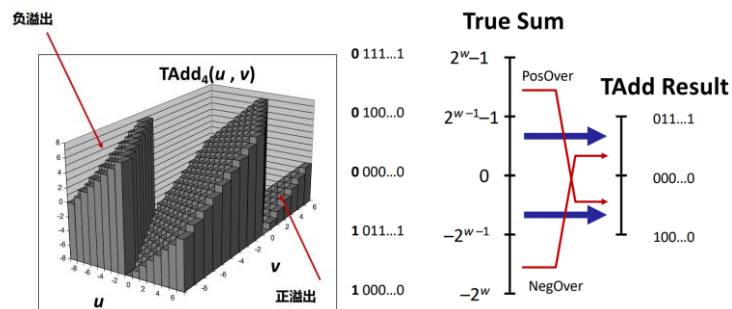
- 无符号数加法——丢弃进位



验证 code:

```
int uadd_ok(unsigned x, unsigned y)
{
    unsigned sum = x+y;
    return sum >= x;
}
```

- 补码加法——将剩余位视为二进制补码整数，操作上一致



验证 code:

```
int tadd_ok(int x, int y)
{
    int sum = x+y;
    int neg_over = x < 0 && y < 0 && sum >= 0;
    int pos_over = x >= 0 && y >= 0 && sum < 0;
    return !neg_over && !pos_over;
}
```

(6) 乘法、除法与移位 —— 分为变量相乘，变量和常量相乘

- 无符号数：最高可为 $2w$ 位

先转化为无符号数，再进行乘法计算，最后转化回补码

补码最小值(负数)：最高可为 $2w-1$ 位

补码最大值(正数)：最高可为 $2w$ 位，仅适用于 $(T_{Min})^2$

- 有符号和无符号的乘法上，**低位是一样的，高位不一样！**

验证 code:

```
int tmult_ok(int x, int y)
{
    int p = x*y; /* Either x is zero, or dividing p by x gives y */
    return !x || p/x == y;
}
```

- 带移位的二次幂乘——编译器自动完成

$$x * 14 = (x \ll 3) + (x \ll 2) + (x \ll 1)$$

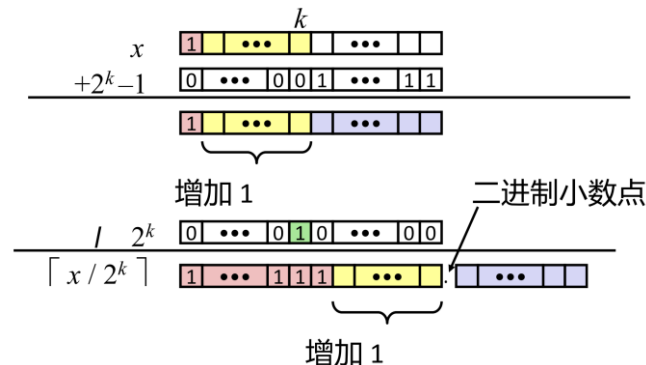
$$x * 14 = (x \ll 4) - (x \ll 1)$$

- 带移位的无符号数 2 次幂除法——逻辑移位

- 带移位的有符号数 2 次幂除法

若采用算术移位，则向下舍入。

若希望向 0 舍入，采用有偏置的二次幂除法。 $(x + (1 \ll k) - 1) \gg k$



注：只要打算右移舍去的位中有一个高位，势必发生进位！

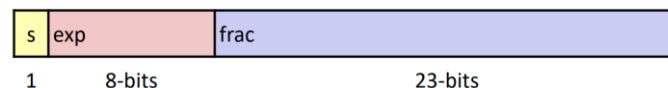
四、浮点数

- 局限 #1: 只能准确地表示形如 $x/2^k$ 的数，其他的有理数有循环的位表示
- 局限 #2: 对于 w 位数，只有一个小数点位置的设定

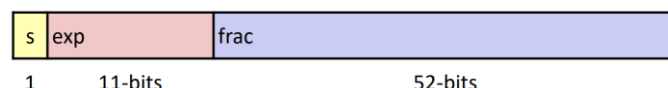
(1) IEEE 浮点标准：定义

- 符号位 s 决定数字是负数还是正数
- 尾数 M 通常是一个范围为 $[1.0, 2.0)$ 内的小数
- 阶码 E 是 2 的幂，表明权重

单精度：32 bits



双精度：64 bits



- 规格化的值 —— $\text{exp} \neq 0$ and $\text{exp} \neq 11 \dots 11$

$$E = \text{exp} - \text{Bias}$$

$$\text{Bias} = 2^{k-1} - 1 \quad \text{单精度: } 127 \quad \text{双精度: } 1023$$

尾数编码，有一个隐含的最高位 1

- 非规格化的值 —— $\text{exp} = 00 \dots 00$

$$E = 1 - \text{Bias} \quad \text{注: 此处不是 } 0 - \text{Bias}$$

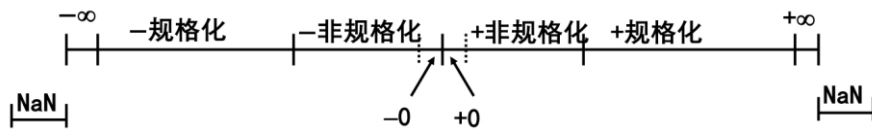
尾数编码，有一个隐含的最高位 0

均匀分布

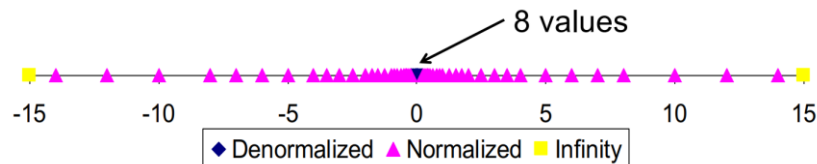
- 特殊值 —— $\text{exp} = 11 \dots 11$

- $\text{frac} = 000 \dots 0$: 表示值 ∞ (无穷大) 或运算结果的溢出

- $\text{frac} \neq 000\dots 0$: Not-a-Number (NaN), 表示无法确定数值的情况



(2) 示例和性质



- 最大的规格化值
单精度 $\approx 3.4 \times 10^{38}$
双精度 $\approx 1.8 \times 10^{308}$
- (几乎) 可以使用无符号整数比较
先比较符号位, 必须考虑 $-0 = +0$, NaN 有点麻烦

(3) 舍入、加法、乘法

【基本思想】

首先计算**精确**结果;
使之**适配**到期望的精度;
如果**指数**太大, 可能会溢出;
可能需要**舍入**来适配**小数 frac** 字段 —— **向偶数舍入法**

$$1.010 \cdot 2^2 + 1.110 \cdot 2^3 = (0.1010 + 1.1100) \cdot 2^3 = 10.0110 \cdot 2^3 = 1.00110 \cdot 2^4 = 1.010 \cdot 2^4$$

注: 如果 E 超出范围, 则溢出

- 加法运算律:
满足交换律、单调性 (除了无穷大和 NaN)
不满足结合律: 如 $(3.14 + 1e20) - 1e20 = 0$, $3.14 + (1e20 - 1e20) = 3.14$
- 乘法运算律:
满足交换律、单调性 (除了无穷大和 NaN)
不满足结合律: 如 $(1e20 \cdot 1e20) \cdot 1e-20 = \text{inf}$, $1e20 \cdot (1e20 \cdot 1e-20) = 1e20$
 $1e20 \cdot (1e20 - 1e20) = 0.0$, $1e20 \cdot 1e20 - 1e20 \cdot 1e20 = \text{NaN}$

(4) C 语言中的浮点数

- int, float, 和 double 之间的转换**会改变位表示**
- double/float $\rightarrow$ int 截断小数部分, 向零舍入, 超出表示范围或 NaN 时, 行为未定义: 通常设置为 T_{Min}
- int $\rightarrow$ double 精确转换, 只要 $\text{int} \leq 53$ 位
- int $\rightarrow$ float 可能会进行舍入

本章总结

- (1) 了解计算机系统中，整数、浮点数的存储的格式.
- (2) 为后面汇编语言的学习提供了理论基础.

整型浮点怎么存？系统 *Bug* 从何来？

《计算机系统基础》重点知识及方法总结

刘鼎琨 于珞珈山

目录

第三章：程序的机器级表示	2
一、 中央处理器的发展历史简述（Central Processing Unit）	2
二、 整体汇编语言体系	3
三、 访问与存储信息	4
四、 算数与逻辑运算	5
五、 控制	5
六、 过程	9
七、 数据结构	11
八、 缓冲区溢出	12
本章总结	14

第三章：程序的机器级表示

一、中央处理器的发展历史简述（Central Processing Unit）

（1）Intel 和 AMD 的渊源：

- ENIAC 计算机，宾夕法尼亚大学，真空管构成。
- 1947 年贝尔实验室，晶体管技术。
- 肖克利博士的“八叛徒”创办了仙童半导体公司。
- 1968 年，Intel 公司成立；1969 年，AMD 公司成立。

（2）早期合作阶段：

- 1971 年，4 位 CPU4004。
- 1972 年，8 位 CPU8008。
- 1978 年，16 位 CPU8086（X86 架构自此而来）。
- 在 IBM(International Business Machines Corporation)的调节下合作。

（3）频率和位宽提升阶段：

■ Intel 公司：

- 1993 年，Intel 公司奔腾 Pentium 32 位 CPU。
- 2000 年，奔腾 4 能达到 4GHz，但是性能提升不大，称为“电火炉”。
- 2001 年，开发 IA64 使用安腾 Itanium 架构，兼容性差。
- 2004 年，转为 X86-64 架构。

■ AMD 公司：

- 收购 NexGen 公司，K6、K7，1999 年 K7 可达 1GHZ。
- 2003 年，速龙 Athlon X86-64 架构。
- 2005 年，速龙 Athlon x2，原生双核 CPU。

（4）多核、大小核发展阶段：

■ Intel 公司：

- 2006 年，酷睿系列 Core 2 双核 CPU。
- 2008 年，Core i7 四核 CPU。
- 直到 2017 年，多线程四核 CPU。
- i5,i7,i9 发展大小核技术，拓展到 16 核。

■ AMD 公司：

- 收购 ATI，重点发展显卡技术，未有成效。
- 2016 年，推出锐龙 Zen 多核系列，后开发锐龙 5000。

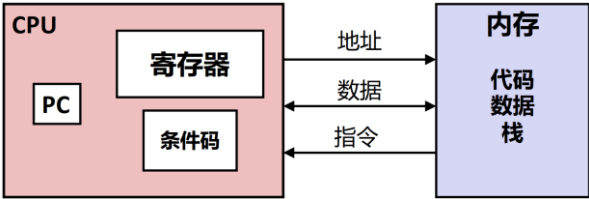
（5）其他正在发展中的 CPU：

- 如苹果的 M1/M2/M3 系列，A 系列等。

注：x86 是复杂指令集计算机架构(CISC)，还有精简指令集计算机架构(RISC)
Complex Instruction Set Computing Reduced Instruction Set Computing

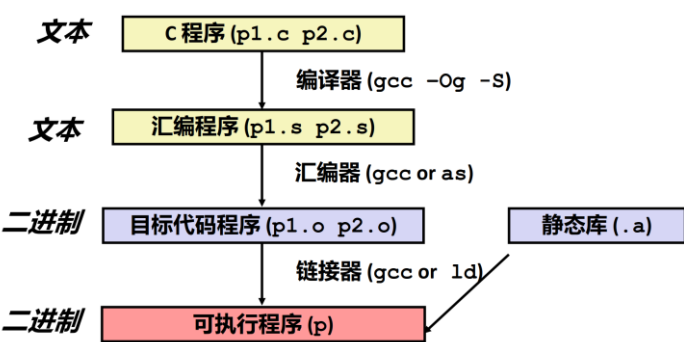
二、 整体汇编语言体系

(1) 最重要的计算机构成图：



注：其中的 PC 特指%rip.

(2) 汇编程序的编译与执行：



注：链接器（也称连接器），用于三个目的：分散再合并源程序；文件调用了某个库的子程序；需要做一些处理才能让程序可执行.

(3) X86-64 框架的寄存器分布：

63	31	15	7	0	
%rax	%eax	%ax	%al		返回值
%rbx	%ebx	%bx	%bl		被调用者保存
%rcx	%ecx	%cx	%cl		第4个参数
%rdx	%edx	%dx	%dl		第3个参数
%rsi	%esi	%si	%sil		第2个参数
%rdi	%edi	%di	%dil		第1个参数
%rbp	%ebp	%bp	%bpl		被调用者保存
%rsp	%esp	%sp	%spl		栈指针
%r8	%r8d	%r8w	%r8b		第5个参数
%r9	%r9d	%r9w	%r9b		第6个参数
%r10	%r10d	%r10w	%r10b		调用者保存
%r11	%r11d	%r11w	%r11b		调用者保存
%r12	%r12d	%r12w	%r12b		被调用者保存
%r13	%r13d	%r13w	%r13b		被调用者保存
%r14	%r14d	%r14w	%r14b		被调用者保存
%r15	%r15d	%r15w	%r15b		被调用者保存

三、访问与存储信息

(1) MOV 指令集:

① 对象:

S	D
立即数	寄存器
	内存
寄存器	寄存器
	内存
内存	寄存器

注: 单个指令无法实现数据的内存到内存传送.

② 具体指令:

指令	效果	描述
MOV S, D	$D \leftarrow S$	传送
movb	$R \leftarrow I$	传送字节
movw		传送字
movl		传送双字
movq		传送四字
movabsq I, R		传送绝对的四字

注: i. movl 特殊, 高 32 位零扩展. **32 位的指令会零扩展到高 32 位.**

ii. movq 特殊, 不能给寄存器赋值 64 位立即数, 需要用到 movabsq.

③ 寻址模式的完整形式: $D(Rb, Ri, S)$ 即 $Mem[Reg[Rb] + S * Reg[Ri] + D]$

- D: 常偏移量, 1、2 或 4 字节
- Rb: 基址寄存器, 16 个整数寄存器中的任何一个
- Ri: 变址寄存器, 除 %rsp 之外的任何寄存器
- S: 比例因子

(2) 零扩展 MOVZ 的指令集:

指令	效果	描述
MOVZ S, R	$R \leftarrow \text{零扩展}(S)$	以零扩展进行传送
movzbw		将做了零扩展的字节传送到字
movzbl		将做了零扩展的字节传送到双字
movzwl		将做了零扩展的字传送到双字
movzbq		将做了零扩展的字节传送到四字
movzwl		将做了零扩展的字传送到四字

注: 此处没有 movzlq 的原因是前表中 movl 已经具备相应的功能.

(3) 符号扩展 MOVS 的指令集:

指令	效果	描述
MOVS S, R	$R \leftarrow \text{符号扩展}(S)$	传送符号扩展的字节
movsbw		将做了符号扩展的字节传送到字
movsbl		将做了符号扩展的字节传送到双字
movswl		将做了符号扩展的字传送到双字
movsbq		将做了符号扩展的字节传送到四字
movswq		将做了符号扩展的字传送到四字
movslq	$\%rax \leftarrow \text{符号扩展}(\%eax)$	将做了符号扩展的双字传送到四字
cltq		把 %eax 符号扩展到 %rax

注: cltq 和 movslq %eax, %rax 完全一致.

四、 算数与逻辑运算

(1) 常用指令集:

指令	效果	描述
<code>leaq S, D</code>	$D \leftarrow \&S$	加载有效地址
<code>INC D</code>	$D \leftarrow D + 1$	加1
<code>DEC D</code>	$D \leftarrow D - 1$	减1
<code>NEG D</code>	$D \leftarrow -D$	取负
<code>NOT D</code>	$D \leftarrow \sim D$	取补
<code>ADD S, D</code>	$D \leftarrow D + S$	加
<code>SUB S, D</code>	$D \leftarrow D - S$	减
<code>IMUL S, D</code>	$D \leftarrow D * S$	乘
<code>XOR S, D</code>	$D \leftarrow D \wedge S$	异或
<code>OR S, D</code>	$D \leftarrow D S$	或
<code>AND S, D</code>	$D \leftarrow D \& S$	与
<code>SAL k, D</code>	$D \leftarrow D \ll k$	左移
<code>SHL k, D</code>	$D \leftarrow D \ll k$	左移 (等同于SAL)
<code>SAR k, D</code>	$D \leftarrow D \gg_A k$	算术右移
<code>SHR k, D</code>	$D \leftarrow D \gg_L k$	逻辑右移

(2) `leaq` 指令集:

注: 要区分取址模式, 本质上理解为做了线性运算.

例:

```
leaq(%rdi,%rdi,2), %rax    # t = x+2*x
salq $2, %rax              # return t<<2 反汇编时不要写成除法.
```

(3) 常见用法: `xorq %rdx, %rdx` # `%rdx = 0`

(4) 移位指令:

注: 其中 `k` 可以是立即数, 也可以是特定的 `%cl` 寄存器, 会根据指令的长度 (如: `salq`, `salw` 等) 从末尾读取相应位数.

(5) 特殊的算术运算:

指令	效果	描述
<code>imulq S</code>	$R[\%rdx] \leftarrow R[\%rax] \leftarrow S \times R[\%rax]$	有符号全乘法
<code>mulq S</code>	$R[\%rdx] \leftarrow R[\%rax] \leftarrow S \times R[\%rax]$	无符号全乘法
<code>cltq</code>	$R[\%rdx] \leftarrow R[\%rax] \leftarrow \text{符号扩展}(R[\%rax])$	转换为八字
<code>idivq S</code>	$R[\%rdx] \leftarrow R[\%rdx] \leftarrow R[\%rax] \bmod S$ $R[\%rax] \leftarrow R[\%rdx] \leftarrow R[\%rax] \div S$	有符号除法
<code>divq S</code>	$R[\%rdx] \leftarrow R[\%rdx] \leftarrow R[\%rax] \bmod S$ $R[\%rax] \leftarrow R[\%rdx] \leftarrow R[\%rax] \div S$	无符号除法

注: `cltq` 的作用更多是在除法之前进行准备.

五、 控制

(1) 条件码的概念:

- **CF: 进位标志**. 最近的操作使最高位产生了进位. 可用来检查无符号操作的溢出.
- **SF: 符号标志**. 最近的操作得到的结果为负数.
- **ZF: 零标志**. 最近的操作得出的结果为 0.
- **OF: 溢出标志**. 最近的操作导致一个补码溢出 -- 正溢出或负溢出.

(2) 隐式设置:

- 常用算数与逻辑指令集中, 除却 `leaq` 之外, 都会设置.
- 比较特殊: `xor` 进位和溢出的标志置 0. 移位操作, 进位为最后被移出的位, 溢出标志置 0.

(3) 显式设置:

指令	基于	描述
CMP S_1, S_2	$S_2 - S_1$	比较
<code>cmpb</code>		比较字节
<code>cmpw</code>		比较字
<code>cmpd</code>		比较双字
<code>cmpq</code>		比较四字
TEST S_1, S_2	$S_1 \& S_2$	测试
<code>testb</code>		测试字节
<code>testw</code>		测试字
<code>testd</code>		测试双字
<code>testq</code>		测试四字

注: `testq %rax, %rax` 可以实现检查 `%rax` 是否为正数, 负数和零.

(4) 显示读取:

指令	同义名	效果	设置条件
<code>sete D</code>	<code>setz</code>	$D \leftarrow ZF$	相等/零
<code>setne D</code>	<code>setnz</code>	$D \leftarrow \sim ZF$	不等/非零
<code>sets D</code>		$D \leftarrow SF$	负数
<code>setns D</code>		$D \leftarrow \sim SF$	非负数
<code>setg D</code>	<code>setnle</code>	$D \leftarrow \sim (SF \wedge OF) \& \sim ZF$	大于 (有符号 >)
<code>setge D</code>	<code>setnl</code>	$D \leftarrow \sim (SF \wedge OF)$	大于等于 (有符号 >=)
<code>setl D</code>	<code>setnge</code>	$D \leftarrow SF \wedge OF$	小于 (有符号 <)
<code>setle D</code>	<code>setng</code>	$D \leftarrow (SF \wedge OF) \mid ZF$	小于等于 (有符号 <=)
<code>seta D</code>	<code>setnbe</code>	$D \leftarrow \sim CF \& \sim ZF$	超过 (无符号 >)
<code>setae D</code>	<code>setnb</code>	$D \leftarrow \sim CF$	超过或相等 (无符号 >=)
<code>setb D</code>	<code>setnae</code>	$D \leftarrow CF$	低于 (无符号 <)
<code>setbe D</code>	<code>setna</code>	$D \leftarrow CF \mid ZF$	低于或相等 (无符号 <=)

注: i. 分清楚有符号和无符号的指令差异.

ii. `set` 和 `cmp` 共同使用时, 逻辑顺序是颠倒的, 需要反过来看.

iii. 基本操作流程: `D` 为 1 个字节大小, 可能需要扩展到 64 位.

(5) 跳转指令:

指令	同义名	跳转条件	描述
<code>jmp Label</code>		1	直接跳转
<code>jmp *Operand</code>		1	间接跳转
<code>jz Label</code>	<code>jz</code>	ZF	相等/零
<code>jne Label</code>	<code>jnz</code>	$\sim ZF$	不相等/非零
<code>js Label</code>		SF	负数
<code>jns Label</code>		$\sim SF$	非负数
<code>jg Label</code>	<code>jnle</code>	$\sim (SF \wedge OF) \& \sim ZF$	大于 (有符号 >)
<code>jge Label</code>	<code>jnl</code>	$\sim (SF \wedge OF)$	大于或等于 (有符号 >=)
<code>jl Label</code>	<code>jnge</code>	$SF \wedge OF$	小于 (有符号 <)
<code>jle Label</code>	<code>jng</code>	$(SF \wedge OF) \mid ZF$	小于或等于 (有符号 <=)
<code>ja Label</code>	<code>jnbe</code>	$\sim CF \& \sim ZF$	超过 (无符号 >)
<code>jae Label</code>	<code>jnb</code>	$\sim CF$	超过或相等 (无符号 >=)
<code>jb Label</code>	<code>jnae</code>	CF	低于 (无符号 <)
<code>jbe Label</code>	<code>jna</code>	$CF \mid ZF$	低于或相等 (无符号 <=)

注: `jmp %rax` 和 `jmp *(%rax)` 代表跳转到相应地址的绝对跳转.

➤ 跳转指令的编码理解:

4003fa: 74 02 je XXXXXX
 4003fc: ff d0 callq *%rax

(6) 分支结构的实现:

① 条件控制实现:

```

t = test-expr;
if (!t)
    goto false;
then-statement
goto done;
false:
    else-statement
done:

t = test-expr;
if (t)
    goto true;
else-statement
goto done;
true:
    then-statement
done:

```

② 条件传送实现:

- 原因: 条件控制有很高的分支预测错误成本, 会打断 CPU 的流水线进程.
- 实现方式:

```

long absdiff(long x, long y)
{
    long result;
    if (x < y)
        result = y - x;
    else
        result = x - y;
    return result;
}

```

```

long absdiff(long x, long y)
x in %rdi, y in %rsi
1  absdiff:
2      movq    %rsi, %rax
3      subq    %rdi, %rax      rval = y-x
4      movq    %rdi, %rdx
5      subq    %rsi, %rdx      eval = x-y
6      cmpq    %rsi, %rdi      Compare x:y
7      cmovge  %rdx, %rax      If >=, rval = eval
8      ret                                Return tval

```

注: 发现之前已经把 rval 存储在 %rax 中.

■ 引入条件传送指令:

指令	同义名	传送条件	描述
cmove S, R	cmovz	ZF	相等/零
cmovne S, R	cmovnz	~ZF	不相等/非零
cmovs S, R		SF	负数
cmovns S, R		~SF	非负数
cmovg S, R	cmovnle	~(SF ^ OF) & ~ZF	大于 (有符号>)
cmovge S, R	cmovnl	~(SF ^ OF)	大于或等于 (有符号>=)
cmovl S, R	cmovnge	SF ^ OF	小于 (有符号<)
cmovle S, R	cmovng	(SF ^ OF) ZF	小于或等于 (有符号<=)
cmova S, R	cmovnbe	~CF & ~ZF	超过 (无符号>)
cmovae S, R	cmovnb	~CF	超过或相等 (无符号>=)
cmovb S, R	cmovnae	CF	低于 (无符号<)
cmovbe S, R	cmovna	CF ZF	低于或相等 (无符号<=)

■ 使用的限制:

- ✧ 风险性: `val = p ? *p : 0;`
- ✧ 不安全性: `val = x > 0 ? x*7 : x+=3;`
- ✧ 计算成本高: `val = Test(x) ? Hard1(x) : Hard2(x);`

(7) 循环结构的实现:

loop:	goto test;	t = test-expr;	init-expr;
body-statement	loop:	if (!t)	while (test-expr) {
t = test-expr;	body-statement	goto done;	body-statement
if (t)	test:	loop:	update-expr;
goto loop;	t = test-expr;	body-statement	}
	if (t)	t = test-expr;	
	goto loop;	if (t)	
		goto loop;	
		done:	

(a) do-while (b) while (c) guarded-do (d) for

注: 虽然 for 向 while 的转化很简单, 但需要考虑 continue 可能存在的问题, 即阻塞索引更新.

(8) switch 结构:

```
void switch_eg(long x, long n,
               long *dest)
{
    long val = x;

    switch (n) {

    case 100:
        val *= 13;
        break;

    case 102:
        val += 10;
        /* Fall through */

    case 103:
        val += 11;
        break;

    case 104:
    case 106:
        val *= val;
        break;

    default:
        val = 0;
    }

    *dest = val;
}
```

```
1  .section      .rodata
2  .align 8      Align address to multiple of 8
3  .L4:
4  .quad .L3      Case 100: loc_A
5  .quad .L8      Case 101: loc_def
6  .quad .L5      Case 102: loc_B
7  .quad .L6      Case 103: loc_C
8  .quad .L7      Case 104: loc_D
9  .quad .L8      Case 105: loc_def
10 .quad .L7      Case 106: loc_D
```

```
void switch_eg(long x, long n, long *dest)
x in %rdi, n in %rsi, dest in %rdx
1  switch_eg:
2  subq $100, %rsi      Compute index = n-100
3  cmpq $6, %rsi        Compare index:6
4  ja   .L8             If >, goto loc_def
5  jmp  *.L4(,%rsi,8)    Goto *jt[index]
6  .L3:                 loc_A:
7  leaq (%rdi,%rdi,2), %rax 3*x
8  leaq (%rdi,%rax,4), %rdi val = 13*x
9  jmp  .L2             Goto done
10 .L5:                 loc_B:
11 addq $10, %rdi        x = x + 10
12 .L6:                 loc_C:
13 addq $11, %rdi        val = x + 11
14 jmp  .L2             Goto done
15 .L7:                 loc_D:
16 imulq %rdi, %rdi      val = x * x
17 jmp  .L2             Goto done
18 .L8:                 loc_def:
19 movl $0, %edi         val = 0
20 .L2:                 done:
21 movq %rdi, (%rdx)     *dest = val
22 ret                  Return
```

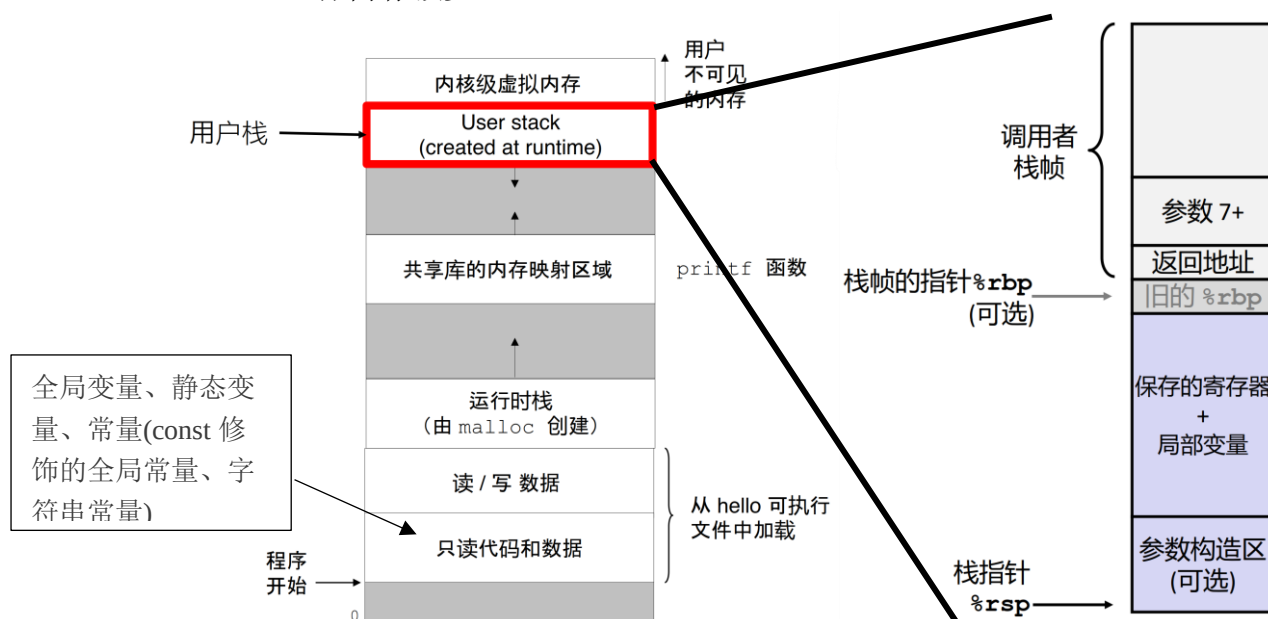
注: i. 在跳转表中多次重复的标记, 可以理解为多 case 合并或 default.

ii. 汇编代码中没有 JMP 指令的默认 fall through.

iii. 使用情况: 当开关数量比较多, 范围跨度小的情况.

六、 过程

(1) 整体内存预览:



注: i. 用户栈的大小很小, 每个程序的栈大小固定(但可调), 像 `push` 和 `pop` 指令都使用的用户栈, 而所有程序共用运行 `malloc` 创建的堆区, 可以动态扩大, 故较大.

ii. 使用栈结构的原因: 寄存器的大小和数量不够, 这是较为安全和规范的方式.

(2) `push()` 和 `pop()` 指令

指令	效果	描述
<code>pushq S</code>	$R[\%rsp] \leftarrow R[\%rsp] - 8;$ $M[R[\%rsp]] \leftarrow S$	将四字压入栈
<code>popq D</code>	$D \leftarrow M[R[\%rsp]];$ $R[\%rsp] \leftarrow R[\%rsp] + 8$	将四字弹出栈

注: i. `pop` 函数的 `D` 必须为寄存器, 其过程内存没有变化, 只有 `%rsp` 的值在改变.

ii. 两个命令均可以由其他基本指令进行合成, 但编码的紧凑度不高.

(3) 转移控制: 对应上图“返回地址”部分.

指令	描述
<code>call Label</code>	过程调用
<code>call *Operand</code>	过程调用
<code>ret</code>	从过程调用中返回

注: i. `call` 命令会将返回地址(当前命令地址的下一命令)压栈, 同时令 `%rip(PC)` 为 `label` 的地址. `ret` 命令则弹出返回地址, 赋值给 `%rip(PC)`, 由此分析可知无需关注具体的压栈和出栈逻辑.

ii. 函数调用时尤其在支持变长的栈帧中可以选用 `%rbp`, 稳定性强.

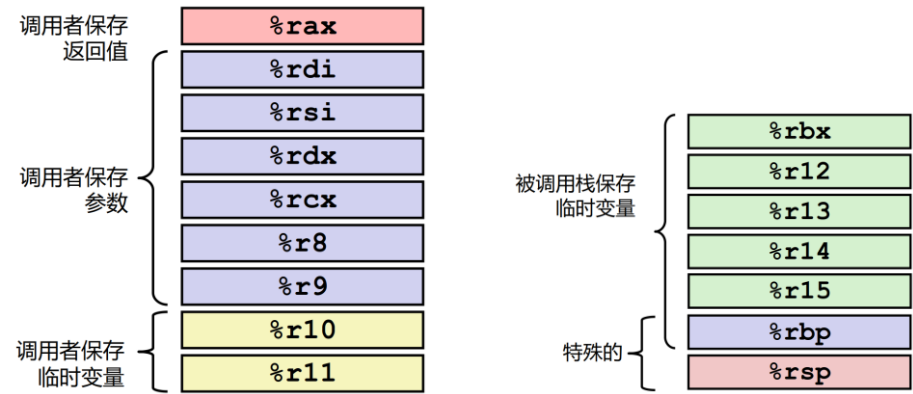
(4) 数据传输: 对应上图“参数 7+, 参数构造区”部分.

操作数大小(位)	参数数量					
	1	2	3	4	5	6
64	%rdi	%rsi	%rdx	%rcx	%r8	%r9
32	%edi	%esi	%edx	%ecx	%r8d	%r9d
16	%di	%si	%dx	%cx	%r8w	%r9w
8	%dil	%sil	%dl	%cl	%r8b	%r9b

注：i. 如果传递的参数是 `int` 类型的，则高 32 位为未定义，不能也不应该调用。
ii. 如果传递的参数多余六个，则需要内存压栈实现，其中参数 7 位于栈顶，所有数据向 8 的倍数对齐。

(5) 局部存储：对应上图“局部变量”部分。
• 什么情况下使用局部内存：寄存器不足以放置所有本地数据，使用“&”必须为之分配地址，局部变量是数组、结构。

(6) 局部存储空间：对应上图“保存的寄存器”部分。



(7) 完整函数的调用：此函数既有参数传入，又在其中递归调用了本函数。

.L1

```
testq    %rdi, %rdi
je       .L6

pushq    %rbp
pushq    %rbx
subq     %8, %rsp

...

movq     %rsi, %rdi
call     .L1
movq     %rax, %rbx

...

addq     %8, %rsp
popq     %rbx
popq     %rbp
```

.L6:

```
rep; ret
```

注：汇编语言的交互性比较差！

递归调用的函数中断条件在函数头部。

保存的寄存器，以及设置栈顶指针。

函数逻辑，可以将部分量压入栈中。

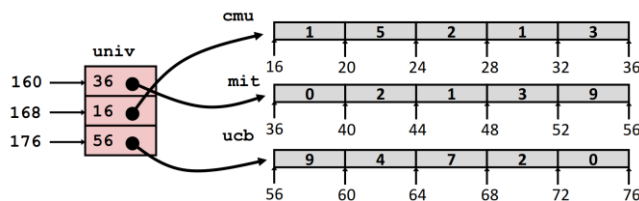
为函数调用做准备，包放置参数，保存相应寄存器。

先恢复栈顶指针，再还原寄存器的初始状态。

七、 数据结构

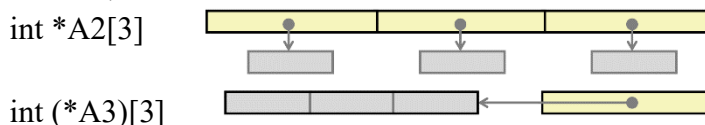
(1) 数组:

- ① 一维数组: $&D[i] = x_D + L * i$;
- ② 二维数组: 若为“行优先”模式, 则: $&D[i][j] = x_D + L * (C * i + j)$;
- ③ 指针数组:



需要寻址两次: $\text{Mem}[\text{Mem}[\text{univ} + 8 * \text{index}] + 4 * \text{digit}]$

注: 注意和数组指针的区别



④ 定长数组: 在宏定义阶段完成数组行列数的声明.

■ 在这里主要讨论其-O1 编译的优化:

```

1  /* Compute i,k of fixed matrix product */
2  int fix_prod_ele_opt(fix_matrix A, fix_matrix B, long i, long k) {
3      int *Aptr = &A[i][0]; /* Points to elements in row i of A */
4      int *Bptr = &B[0][k]; /* Points to elements in column k of B */
5      int *Bend = &B[N][k]; /* Marks stopping point for Bptr */
6      int result = 0;
7      do {
8          result += *Aptr * *Bptr; /* Add next product to sum */
9          Aptr++; /* Move Aptr to next column */
10         Bptr += N; /* Move Bptr to next row */
11     } while (Bptr != Bend); /* Test for stopping point */
12     return result;
13 }

```

⑤ 变长数组: 数组的行与列为变量.

和定长数组最大的差别是需要用到乘法对 i 伸缩 n 倍.

注: 一个变量和一个常数的乘法一般使用移位和加法进行处理, 而两个变量的乘法只能使用 `imulq` 乘法指令.

(2) 结构体:

■ 结构对齐的原则:

- 基本数据类型需要 K 字节表示, 则地址必须能是 K 的倍数
- 最大的对齐要求 K , 整体结构必须是 K 的倍数

■ 结构对齐的空间节省: 把大型数据类型放在首位

注: 元素是结构体的数组, 则整体结构长度乘以 K , 满足每个元素的对齐要求. 结构体中包含结构体的情形类似.

(3) 联合体:

```
typedef union {
    struct {
        long    u;
        short   v;
        char     w;
    } t1;
    struct {
        int a[2];
        char *p;
    } t2;
} u_type;

void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

u		v	w					
a[0]	a[1]	p						

表达式	类型	代码
up->t1.u	long	movq(%rdi),%rax movq %rax, (%rsi)
up->t1.v	short	movw 8(%rdi),%ax movw %ax, (%rsi)
&up->t1.w	char*	addq \$,%rdi movq %rdi, (%rsi)
up->t2.a	int*	movq %rdi,%rsi
up->t2.a[up->t1.u]	int	movq (%rdi),%rax movl (%rdi,%rax,4),%eax movl %eax, (%rsi)
*up->t2.p	char	movq 8(%rdi),%rax movb (%rax),%al movb %al, (%rsi)

八、 缓冲区溢出

- (1) 常见形式：未检查输入字符串的长度，特别是对于栈上有界的字符数组，有时也被称为栈崩溃.
- gets: 输入任意长度的字符数组，直到“EOF”或者“\n”.
- strcpy, strcat: 复制任意长度的字符串.
- scanf, fscanf, sscanf 当给定 %s 转换规范时.
- (2) 以 gets 函数为例的缓冲区溢出:

栈帧 call_echo			
00	00	00	00
00	40	06	c3
00	32	31	30
39	38	37	36
35	34	33	32
31	30	29	28
27	26	25	24
23	22	21	20

栈帧 call_echo			
00	00	00	00
00	40	00	34
33	32	31	30
39	38	37	36
35	34	33	32
31	30	29	28
27	26	25	24
23	22	21	20

栈帧 call_echo			
00	00	00	00
00	40	06	00
33	32	31	30
39	38	37	36
35	34	33	32
31	30	29	28
27	26	25	24
23	22	21	20

注：如果冲撞了返回地址，可能会“阴差阳错”的不受影响。

(3) 字符串注入攻击：

将字符串通过 `gets` 等函数放入栈中，覆盖了原有上方正确的返回地址，指向攻击代码。

注：蠕虫（worm）指的是一个程序，他可以自己运行，可以将自身的完整工作版本传播到其他计算机；病毒（virus）则是一些代码，将自身添加到其他程序。

(4) 处理缓冲区溢出攻击的几种方法：

① 改用更加安全的函数进行调用：

- 使用 `fgets` 而不是 `get`
- 使用 `strncpy` 而不是 `strcpy`
- 不要将 `scanf` 与 `%s` 转换规范一起使用
 - ✧ 使用 `fgets` 读取字符串或使用 `%ns`，其中 `n` 是合适的整数

② 栈随机化：

- 这里的栈指的是“用户栈”。
- 随机栈虽然不能阻止像 `gets` 函数一样覆盖返回地址，但可以使返回地址无法精确定位到攻击程序上。
- 这种方法对应的偏移量有一定的界限，不能无限制的扩大偏移量。

③ 不可执行的代码段：将一部分的栈区域标记为“不可执行段”。

④ 栈金丝雀：

```

void echo()
1  echo:
2      subq    $24, %rsp           Allocate 24 bytes on stack

3      movq    %fs:40, %rax        Retrieve canary
4      movq    %rax, 8(%rsp)       Store on stack
5      xorl    %eax, %eax          Zero out register
6      movq    %rsp, %rdi          Compute buf as %rsp
7      call    gets                Call gets
8      movq    %rsp, %rdi          Compute buf as %rsp
9      call    puts                Call puts
10     movq    8(%rsp), %rax        Retrieve canary
11     xorq    %fs:40, %rax        Compare to stored value
12     je      .L9                 If =, goto ok
13     call    __stack_chk_fail    Stack corrupted!
14 .L9:                               ok:
15     addq    $24, %rsp           Deallocate stack space
16     ret
  
```

注：`%fs:40` 代表着从内存地址为 40 的地方取出相应长度的数据，相当于随机化生成一段“金丝雀”。

(5) 还存在的一些难题：

- Gadget 方法；
- 形成 ROP（Return-Oriented Programming）链
- 肯定可以破坏掉程序，但不一定可以实现其他目的。
- 本方法可以用金丝雀来保护。

栈帧 for call echo			
00	00	00	00
00	40	04	dc
00	00	00	00
00	40	04	d4
33	32	31	30
39	38	37	36
35	34	33	32
31	30	29	28
27	26	25	24
23	22	21	20

← %rsp

buf

本章总结

- (1) 对于计算机系统的构成有更深入的理解.
- (2) 对汇编语言的语法、逻辑功能进行初步实现.
- (3) 对于优化的算法有一定的了解.
- (4) 其中浮点数相关的指令未能包含进来，确有遗憾.

字节流中寻真意，汇编细语逻辑魂。

《计算机系统基础》重点知识及方法总结

刘鼎琨 于珞珈山

目录

- 第七章：链接 2
 - 一、 链接器的作用与由来 2
 - 二、 典型程序的转换处理过程 2
 - 三、 目标文件 ELF 格式 2
 - 四、 汇编器产生.o 符号表 3
 - 五、 链接器之符号解析 4
 - 六、 链接器之重定位 4
 - 七、 程序加载到内存 5
 - 八、 静态库 5
 - 九、 动态库（共享目标文件） 5
 - 十、 库打桩/插桩 6
- 本章总结 6

第七章：链接

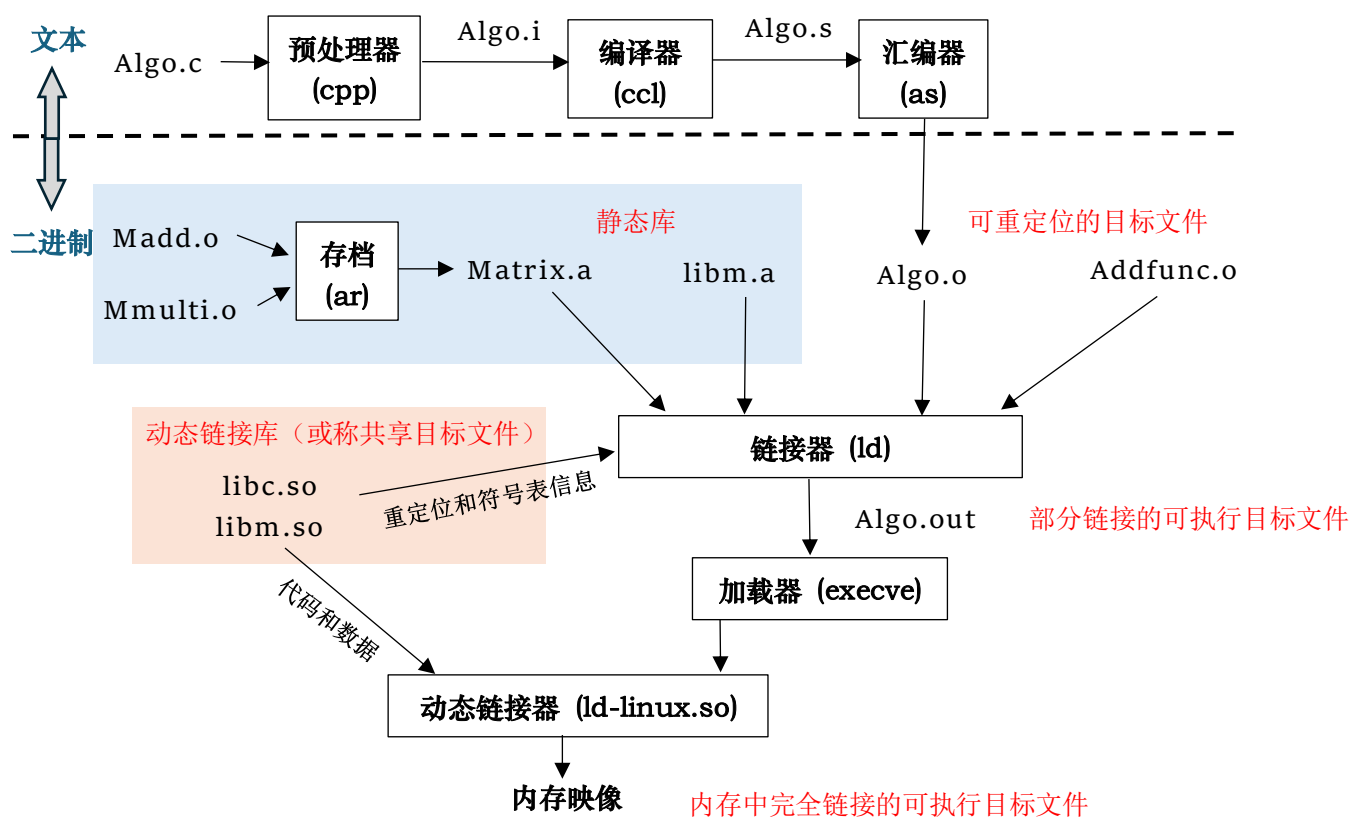
一、 链接器的作用与由来

(1) 作用：

- ① 模块化编程.
- ② 时间效率的提升：可以分开编译，修改一个源文件不需要重新编译其他.
- ③ 空间效率的提升：无需包含共享库的所有代码.

(2) 由来：用机器语言编写程序，需要打印在纸带或卡片上，修改地址需要重新打印，由此推动汇编语言产生，重定位的需求也推动了链接器的发展.

二、 典型程序的转换处理过程



注：在 Windows 系统中，动态链接库为.dll，可执行程序为.exe.

三、 目标文件 ELF 格式

—— Executable and Linkable Format

(1) 目标文件的范围：

- 可重定位目标文件 (.o 文件)
 - 每个.o 文件代码和数据地址都从 0 开始
- 可执行目标文件 (a.out 文件)
 - 代码和数据地址为虚拟地址空间中的地址，但是是绝对的地址.
- 共享目标文件 (.so 文件)
- 静态链接库 (.a 文件)

(2) ELF 文件分区:

- ① ELF header: 魔数 ELF(7F 45 4C 46)、字长、字节序、文件类型等.
- ② 程序头部表: 页大小、虚拟地址内存段 (节, sections)、段大小等, 考虑如何映射, 可执行文件和动态链接库都有此版块.
- ③ .text 节: 代码节, 可读/可执行.
- ④ .rodata 节: 只读数据, 如跳转表.
- ⑤ .data 节: 初始化的全局变量, 可读/可写.
- ⑥ .bss 节: 未初始化的静态变量, 或初始化为 0 的全局和静态变量. 可读/可写. 具有节头部, 但是实际不占用空间.
注: 区分 COMMON 区, 隐藏在 ELF 文件中, 保存了未初始化的全局变量.
- ⑦ .symtab 节: 符号表.
- ⑧ .rel.text 节、.rel.data 节: 包含相关重定位信息.
- ⑨ .debug 节: 有关符号调试的信息.
- ⑩ 节头部表: 每个节的偏移量和大小.

ELF header
Program header table (required for executables)
.text section
.rodata section
.data section
.bss section
.symtab section
.rel.txt section
.rel.data section
.debug section
Section header table

四、 汇编器产生.o 符号表

(1) 全局符号: 由模块内部定义, 能够被其他模块引用的符号.

(2) 外部符号: 模块引用的由其他模块定义的全局符号.

注: 此处正常的调用形式有二: `int func();` `extern int x;`

(3) 局部符号: 由模块独占地定义和引用的符号.

注: i. 如以 `static` 属性定义的 C 函数、变量. 可以通过加上 `static` 使变量同其他文件隔离, 即使其他文件用 `extern` 进行调用.

ii. 局部变量不等同于局部符号, 其位置分配至用户栈区.

iii. 在符号表中为重名的局部符号创建不同的、唯一的名字, 如 `x.1`, `x.2`.

(4) 符号表和 ELF 文件的关系:

■ 符号表 (symtab) 中每个条目的结构如下 (64位系统)

```
typedef struct {
    int name; /*指向符号对应字符串在strtab节中的偏移*
    char type: 4, /*符号对应目标的类型: 数据、函数、源文件、节*/
    binding: 4; /*符号对应目标是全局符号还是局部符号*/
    char reserved;
    short section; /*符号对应目标所在的section, 或其他情况*/
    int value; /*在对应section中的偏移量, 可执行文件中是虚拟地址*/
    int size; /*符号对应目标所占字节数*/
} Elf64_Symbol;
```

`readelf -s <目标文件名>`

其他情况: ABS表示不该被重定位; UND表示未定义; COM表示未初始化数据

(.bss), 此时, value表示对齐要求, size给出最小长度

swap.o 的符号表

Symbol table '.symtab' contains 12 entries:							
Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	swap.c
2:	0000000000000000	0	SECTION	LOCAL	DEFAULT	1	
3:	0000000000000000	0	SECTION	LOCAL	DEFAULT	3	
4:	0000000000000000	0	SECTION	LOCAL	DEFAULT	5	
5:	0000000000000000	0	SECTION	LOCAL	DEFAULT	7	
6:	0000000000000000	0	SECTION	LOCAL	DEFAULT	8	
7:	0000000000000000	0	SECTION	LOCAL	DEFAULT	6	
8:	0000000000000000	8	OBJECT	GLOBAL	DEFAULT	3	bufp0
9:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	buf
10:	0000000000000008	8	OBJECT	GLOBAL	DEFAULT	COM	bufp1
11:	0000000000000000	60	FUNC	GLOBAL	DEFAULT	1	swap

五、 链接器之符号解析

- (1) 目的：将每个模块中引用的符号与某个目标模块中的定义符号建立关联。
- (2) 次序：当每个定义符号在代码段或数据段中都被分配了存储空间后，将引用符号与对应定义符号建立关联，就可在重定位时将引用符号的地址重定位为相关联的定义符号的地址。
- (3) 符号类型：
 - 强符号：过程和初始化了的全局符号，视为定义。
 - 弱符号：未初始化的全局符号，可视为声明。
 注：此处可以体现 COMMON 区的优势，汇编器将决定权交给链接器。
- (4) 解读规则：
 - 规则 1：不允许有多个同名的强符号。
 - 规则 2：如果有一个强符号和多个弱符号，选择强符号。
 - 注：可能会向上覆盖，上方的变量无论顺序都会被覆盖。
 - 规则 3：如果有多个弱符号，任意选择一个。
- (5) 编程建议：尽量避免全局变量；尽量使用 static；尽量初始化好全局变量；用好 extern 等标志。

六、 链接器之重定位

- (1) 重定位的理解：把.o 文件中的相对位置，变为虚拟地址上的绝对内存位置。
- (2) 步骤 1：节的合并及符号地址的确定

多个目标文件和系统代码(.text)、数据(.data)进行合并。

注：i. 由此可以看出，即使没有使用任何库函数，链接的过程也必不可少。
 ii. .bss 节中的局部静态变量也需要在合并的可执行文件中体现。
- (3) 步骤 2：符号引用的重定位

新的代码节和数据节中引用符号的重定位，需要利用 .rel.data 和 .rel.text。

➢ 重定位结构体信息如下：offset、type、symbol、addend。

offset: 节偏移
 type: R_386_PC32 PC 相对寻址(跳转表等); R_386_32 绝对寻址(固定内存).
 symbol: 符号表示.
 addend: 常数偏移量(一般为负)

```

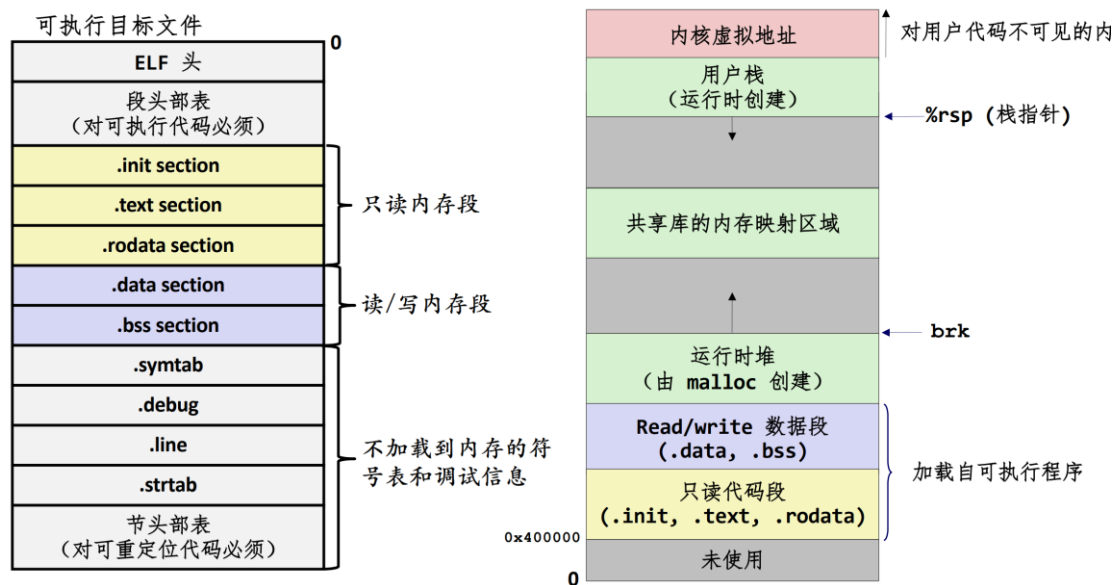
00000000004004d0 <main>:
4004d0: 48 83 ec 08      sub    $0x8,%rsp
4004d4: be 02 00 00 00   mov    $0x2,%esi
4004d9: bf 18 10 60 00   mov    $0x601018,%edi # %edi = &array
4004de: e8 05 00 00 00   callq 4004e8 <sum>    # sum()
4004e3: 48 83 c4 08      add    $0x8,%rsp
4004e7: c3              retq

00000000004004e8 <sum>:
4004e8: b8 00 00 00 00   mov    $0x0,%eax

Relocation section '.rel.text' at offset 0x1e8 contains 2 entries:
Offset          Info          Type          Sym. Value      Sym. Name + Addend
0000000000000a  00090000000a  R_X86_64_32   0000000000000000 array + 0
0000000000000f  000a00000002  R_X86_64_PC32 0000000000000000 sum - 4
  
```

- 相对寻址的计算方法：
- ① $\text{refAddr} = \text{main.address} + \text{rela.offset} = 0x4004d0 + 0xf = 0x4004df$
 - ② 执行 callq, $\text{PC} = \text{refAddr} - \text{rela.addend} = 0x4004df - (-4) = 0x4004e3$
 - ③ 重定位: $\text{sum.addr} - \text{PC} = 0x4004e8 - 0x4004e3 = 0x5$
 - ④ 按照小端模式进行填写。 注：PIC 为动态寻址的方式，常用。

七、 程序加载到内存



注：加载到内存的时机一般是程序运行的时候，程序运行结束后，系统会按照一定规定进行空间释放。

八、 静态库

- (1) 定义：将一些相关联的可重定位目标文件连接成一个带索引的文件（称为存档文件 archive）
- (2) 命令行链接的顺序很重要：链接器按命令行顺序扫描 .o 文件和 .a 文件
 - 原则：将库放在命令行的最后、被引用者放置在引用者后面。
- (3) 劣势：
 - 存储的可执行文件中有大量重复（每个函数都需要 libc）
注：关注的是代码和数据在磁盘存储上的冗余。
 - 运行的可执行文件中有大量重复
注：关注的是代码和数据在运行时内存中的冗余。
 - 对系统库的很小的 bug 修复，要求每个应用程序都进行重新链接

九、 动态库（共享目标文件）

- 可以在可执行文件初次加载和运行时进行动态链接 (load-time linking)
 - ✧ Linux 上最常见的情况，由动态链接器 (ld-linux.so) 自动处理
 - ✧ 标准 C 库 (libc.so) 通常是动态链接的
- 可以在程序开始执行后进行动态链接 (run-time linking)
 - ✧ 在 Linux 上，是通过调用 dlopen() 接口来实现的


```
handle = dlopen("./libvector.so", RTLD_LAZY); //打开共享库
addvec = dlsym(handle, "addvec"); //获得函数体指针
addvec(x, y, z, 2); //正常调用其中的函数
```
 - ✧ § 分布式软件
 - ✧ § 高性能 web 服务器
 - ✧ § 运行时库打桩
- 可以多个进程共享库函数

十、 库打桩/插桩

(1) 库打桩定义：一种强大的链接技术，使得程序员可以截获对任意函数的调用。

(2) 编译时打桩：需要.c 文件.

✧ `#define malloc(size) mymalloc(size)`

✧ `#define free(ptr) myfree(ptr)`

将对 `malloc/free` 的调用宏扩展为对 `mymalloc/myfree` 的调用

(3) 链接时打桩：需要.o 文件.

✧ `gcc -Wall -Wl,--wrap,malloc -Wl,--wrap,free -o intl`

—— 使用链接器的技巧做特殊的名字解析，可以在 `__wrap_free` 内部使用 `__real_free` 来调用真正的 `free` 函数。

相当于：`Malloc` → `__wrap_malloc`

`__real_malloc` → `malloc`

(4) 加载/运行时打桩：需要.out 文件.

利用 `LD_PRELOAD` 环境变量告诉动态链接器在解析未被解析的引用时(例如，对 `malloc` 的引用)，先在自定义的 `mymalloc.so` 中查找。

本章总结

- (1) 理解链接器将帮助你构造大型程序.
- (2) 理解链接器将帮助你避免一些危险的编程错误.
- (3) 理解链接将帮助你理解语言的作用域规则是如何实现的.
- (4) 理解链接将帮助你理解其他重要的系统概念.
- (5) 理解链接将使你能够利用共享库.

——借用教材的概括

打桩置换显踪迹，以动替静妙处多。